

MACHINE LEARNING

Fungi Image Classification

CRUZ URBINA

CSCI 472

5/4/26

Contents

Abstract.....	3
Introduction	4
Exploratory Data Analysis (EDA)	5
Methodology	7
Results and Discussion.....	10
Conclusion	15

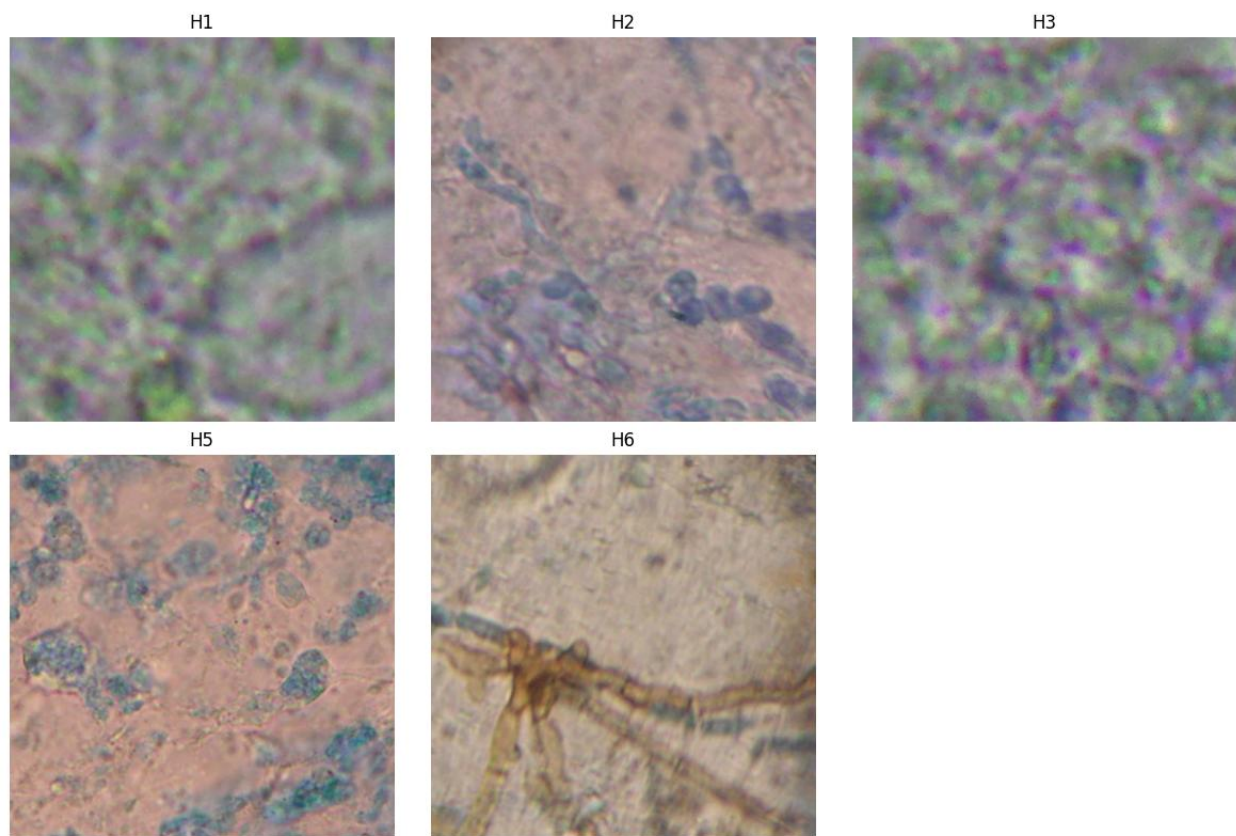
Abstract

This project investigated the use of machine learning to classify microscopic fungi images from the DeFungi dataset. The problem addressed was a five-class image classification task in which each image had to be assigned to one of the labeled categories: H1, H2, H3, H5, or H6. The original dataset contained 9,114 labeled images and showed strong class imbalance, so preprocessing was an important part of the workflow. To prepare the data, image paths were loaded into a structured dataframe, images were resized to 32 x 32, converted to RGB, normalized, flattened into numerical feature vectors, and balanced through augmentation. The labels were encoded, the data was split into training, validation, and test sets, standardized, and reduced in dimensionality using Principal Component Analysis (PCA), which lowered the feature space from 3,072 to 425 components while preserving 95% of the variance.

Several machine learning models were then compared, including LightGBM, XGBoost, Random Forest, Bagging, Decision Tree, and Nearest Centroid. Hyperparameter tuning was performed using randomized search with cross-validation. Among the tuned models, LGBMClassifier produced the best overall performance, achieving about 76% accuracy and a weighted F1 score of about 0.75 on the test set. These results show that classical machine learning methods, when combined with preprocessing, balancing, and dimensionality reduction, can perform effectively on multiclass fungi image classification.

Introduction

This project aims to develop a machine learning model capable of accurately detecting and classifying fungal infections in plants or biological samples using image data. It utilizes the DeFungi dataset, which was specifically created to support the development of classification algorithms for fungal image analysis. The dataset consists of microscopic images obtained from direct mycological examinations of superficial fungal infections caused by yeasts, molds, and dermatophytes. Each image was manually labeled into one of five classes with the assistance of a subject matter expert, and automated algorithms were applied to crop images to their regions of interest. The dataset contains 9,114 instances and is designed for classification tasks using real-valued image features. Prior to modeling, preprocessing steps were applied to ensure consistency and focus on relevant visual information.



(Image 1. Sample images by class)

This project builds a complete image analysis pipeline that loads and organizes the dataset, examines class distribution, and applies preprocessing and data augmentation techniques. The images are then converted into numerical features suitable for machine learning. After scaling the data and reducing dimensionality using Principal Component Analysis (PCA), multiple classification models are trained, compared, and tuned to identify the most effective

approach. Overall, this work demonstrates how supervised learning can enable faster and more consistent classification of fungal images.

Problem Definition

This project addresses a multiclass image classification problem. Given a microscopic image of a fungal sample, the goal is to assign it to one of five categories: H1, H2, H3, H5, or H6. This task is important because manual identification of fungal infections from images can be time-consuming and subject to variability in human interpretation. An accurate and reliable model can improve both efficiency and consistency in fungal image analysis.

Dataset Description

The DeFungi dataset contains 9,114 microscopic images collected for direct mycological examination. These images represent superficial fungal infections caused by yeasts, molds, and dermatophytes, and were labeled with expert guidance. The data is distributed across five classes. To prepare the dataset for modeling, image paths are organized by class, and images are resized and normalized. Data augmentation techniques are applied to underrepresented classes to address class imbalance and improve model performance.

Objective

The primary objective of this project is to develop and evaluate a machine learning model that can accurately classify fungal images into their respective categories. This involves training multiple baseline models, performing hyperparameter tuning on the most promising candidates, and selecting the best-performing model based on evaluation metrics such as the weighted F1 score. In other words, the project seeks to assess whether traditional machine learning techniques, combined with image preprocessing and dimensionality reduction, can effectively solve a real-world multiclass fungi classification problem.

Exploratory Data Analysis (EDA)

The exploratory data analysis (EDA) phase provided key insights into the structure of the dataset and highlighted potential challenges for model development. The DeFungi dataset contains 9,114 microscopic images distributed across five classes: H1, H2, H3, H5, and H6. The class distribution is notably imbalanced, with H1 comprising approximately 48% of the data, followed by H2 at 26%, while H3, H5, and H6 each account for less than 10%. The target variable is not balanced. This imbalance likely reflects real-world fungal prevalence rather than labeling errors, as the dataset was curated with expert input.

Class imbalance introduces several risks, including bias toward the majority class and misleadingly high overall accuracy if used as the sole evaluation metric. To mitigate these issues, class weighting was applied during model training to emphasize minority classes. In addition, performance was evaluated using precision, recall, and F1-score to ensure a more balanced

assessment across all categories. Data augmentation was also used to improve representation of underrepresented classes without discarding valuable data.

Further inspection of the dataset confirmed the presence of 9,114 labeled images organized consistently across the five classes. Visualizing sample images from each category helped verify label accuracy and provided insight into the classification task. While the images were generally consistent, some classes appeared visually similar, suggesting that the model would need to distinguish subtle patterns, increasing the difficulty of the task.

Another key finding was the high dimensionality of the image data. Each image was resized to 32×32 pixels and converted to RGB format, resulting in 3,072 features per image after flattening. This large number of features introduces redundancy, as neighboring pixels are often highly correlated. A full correlation analysis of all pixel values would be impractical due to its size and limited interpretability.

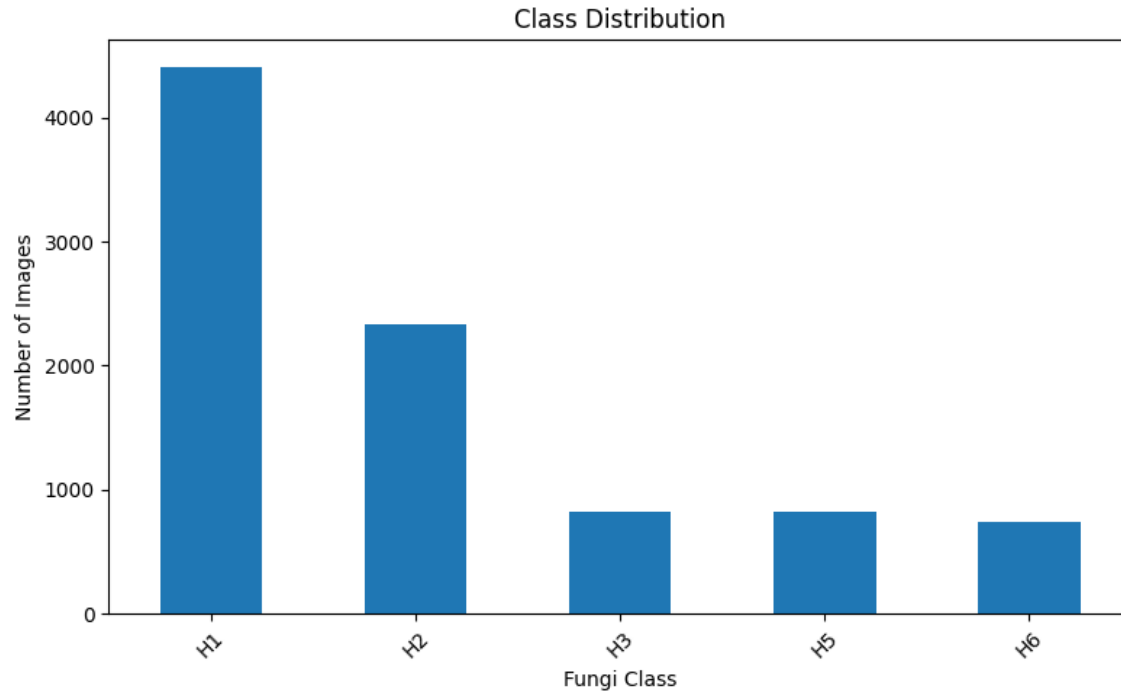
To address this, Principal Component Analysis (PCA) was applied after feature scaling. PCA reduced the dimensionality from 3,072 to 425 features while preserving 95% of the variance. This demonstrated that most of the meaningful information in the images could be retained in a significantly smaller feature space, improving computational efficiency and model performance.

Based on these observations, several challenges were anticipated. The initial class imbalance could bias model predictions if not properly addressed. The high dimensionality of the data required effective preprocessing and dimensionality reduction. Additionally, visual similarity between some classes could make accurate classification more difficult. It was also noted that performing data augmentation before splitting the dataset might introduce similar images across training and testing sets, potentially inflating performance metrics.

To balance the dataset, augmentation was applied to minority classes until all classes contained an equal number of images, increasing the dataset size to 22,020 images. Undersampling was not used, as it would remove a substantial portion of data from majority classes and potentially weaken the model. Instead, combining augmentation with class weighting preserved all available information while improving class balance.

From a computational standpoint, processing the full dataset at once led to memory limitations and runtime instability. To address this, a batch-based loading strategy was implemented, allowing images to be processed in smaller groups. Additionally, images were resized before flattening for traditional machine learning models, reducing memory usage while preserving essential features.

Overall, the EDA phase revealed both the strengths and limitations of the dataset. While the data is sufficiently large and well-structured for machine learning, challenges such as class imbalance, high dimensionality, and visual similarity between classes required careful handling. These insights guided the preprocessing strategy and informed the modeling approach used in the remainder of the project.



(Figure 1. Class Distribution)

Methodology

The methodology followed a structured pipeline to transform raw image data into a format suitable for machine learning and to ensure reliable model development and evaluation. The process began by organizing the DeFungi dataset into a structured dataframe containing image file paths and their corresponding class labels. Since the dataset consists of images rather than tabular data, traditional missing-value imputation was not required. Instead, preprocessing focused on converting images into numerical features. Only valid image formats (.jpg, .jpeg, .png) were included to avoid processing errors. Each image was resized to 32×32 pixels, converted to RGB format, normalized to a $[0, 1]$ range by dividing pixel values by 255, and flattened into a one-dimensional vector of 3,072 features. This ensured consistent input dimensions and prevented scale-related issues during model training. Error handling was incorporated during image loading to skip corrupted or unreadable files without interrupting the workflow. Class labels were encoded using LabelEncoder to convert categorical labels into a numeric format compatible with machine learning models.

	Model	Accuracy	Precision Weighted	Recall Weighted	F1 Weighted
0	LGBMClassifier	0.706140350877193	0.6986590367627916	0.706140350877193	0.7002075802164618
1	XGBClassifier	0.6699561403508771	0.6481864362639959	0.6699561403508771	0.630515038832413
5	BaggingClassifier	0.6480263157894737	0.6296962468643303	0.6480263157894737	0.5977705959702864
3	DecisionTreeClassifier	0.5844298245614035	0.5954122748008226	0.5844298245614035	0.5888779600034033
2	RandomForestClassifier	0.6041666666666666	0.5645970006275504	0.6041666666666666	0.5673200882702627
4	NearestCentroid	0.3892543859649123	0.5160086940755485	0.3892543859649123	0.4104871917848858

(Table 1. Models scores before data augmentation)

To address class imbalance, data augmentation was applied to the minority classes using transformations such as horizontal flipping, slight rotations, and brightness adjustments. This process increased the number of samples in underrepresented classes until all classes contained 4,404 images, resulting in a balanced dataset of 22,020 observations. Undersampling was avoided to prevent the loss of valuable information from majority classes. In addition, class weighting was incorporated during model training to further mitigate imbalance effects.

Given the high dimensionality of the flattened image data, feature scaling and dimensionality reduction were essential. The features were first standardized using StandardScaler to ensure they were centered and comparable, which is particularly important for algorithms sensitive to feature scale. Principal Component Analysis (PCA) was then applied with a variance retention threshold of 95%, reducing the feature space from 3,072 to 425 components. This step removed redundancy, improved computational efficiency, and helped enhance model generalization without significant loss of information.

	Model	Accuracy	Precision Weighted	Recall Weighted	F1 Weighted
0	LGBMClassifier	0.759763851044505	0.7556306979207115	0.759763851044505	0.7530082471007681
1	XGBClassifier	0.7397820163487738	0.7386702494679538	0.7397820163487738	0.7287931947708786
2	RandomForestClassifier	0.6798365122615804	0.6716226507089309	0.6798365122615804	0.6599466814119489
3	BaggingClassifier	0.6553133514986376	0.6499163536341581	0.6553133514986376	0.6283875962885012
4	DecisionTreeClassifier	0.6121707538601272	0.6134908232878056	0.6121707538601272	0.6035932279620766
5	NearestCentroid	0.4736603088101725	0.469898274039826	0.4736603088101725	0.4587290565697413

(Table 2. Models scores after data augmentation)

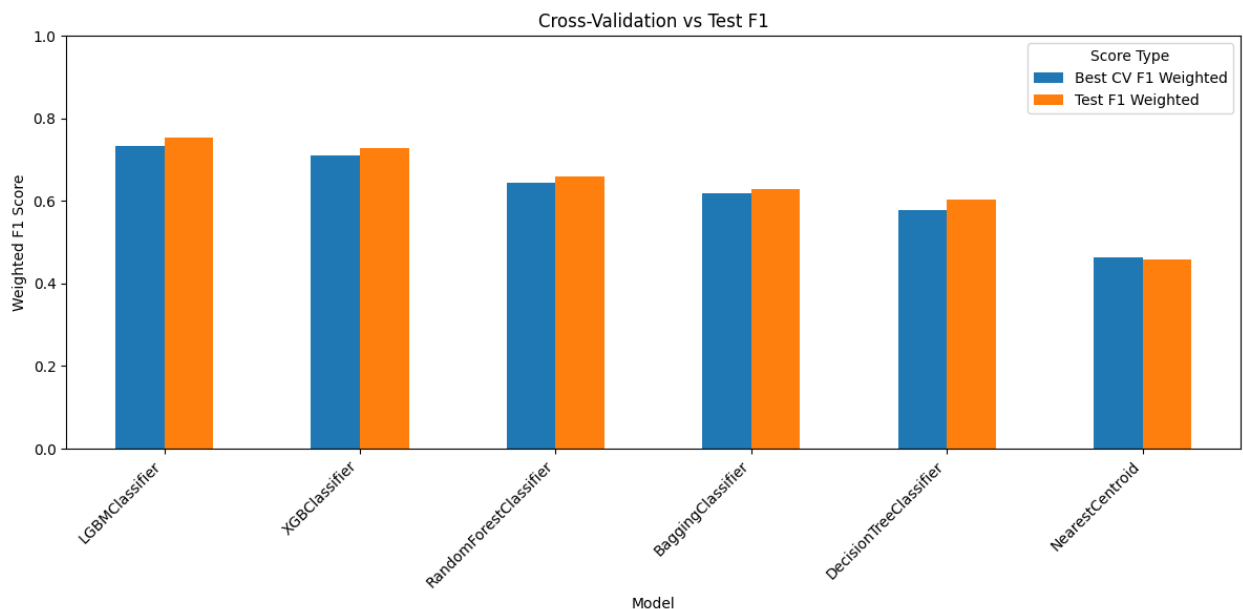
The dataset was split using stratified sampling to maintain class distribution across subsets. An 80/20 split was first applied to create training and temporary holdout data, and the holdout portion was then divided equally into validation and test sets, resulting in an approximate 80/10/10 split. Although a validation set was created, model tuning primarily relied on cross-validation within the training data to ensure robustness.

For model selection, an initial screening was conducted using LazyClassifier to quickly benchmark a wide range of algorithms. Based on this evaluation, six models were selected for further analysis: LGBMClassifier, XGBClassifier, RandomForestClassifier, DecisionTreeClassifier, NearestCentroid, and BaggingClassifier. These models represent a diverse set of learning approaches, including gradient boosting, ensemble tree methods, and distance-based classification, allowing for a comprehensive comparison of performance.

Hyperparameter tuning was performed using RandomizedSearchCV with 3-fold stratified cross-validation and the weighted F1-score as the primary evaluation metric. This approach was chosen for its efficiency in exploring a wide range of parameter combinations without the computational cost of exhaustive grid search. Key parameters tuned included the number of estimators, learning rate, maximum tree depth, and sampling configurations, depending on the model. A second, more focused search was conducted for LightGBM, which emerged as a strong candidate, further refining its performance.

Model evaluation was conducted using multiple metrics, including accuracy, weighted precision, weighted recall, and weighted F1-score. While accuracy provides an overall measure of correctness, it can be misleading in imbalanced datasets. Therefore, greater emphasis was placed on the weighted F1-score, which balances precision and recall while accounting for class distribution. Additionally, a confusion matrix was used to analyze class-level performance and identify patterns of misclassification. From a computational perspective, processing the full dataset at once led to memory limitations. To address this, a batch-based loading approach was

implemented, allowing images to be processed in smaller subsets. Resizing images before flattening also helped reduce memory usage while preserving essential visual features. Overall, this methodology systematically converts raw image data into a structured format, reduces dimensionality, addresses class imbalance, and applies robust model selection and evaluation techniques. These steps ensure that the resulting models are both efficient and reliable for multiclass fungal image classification.



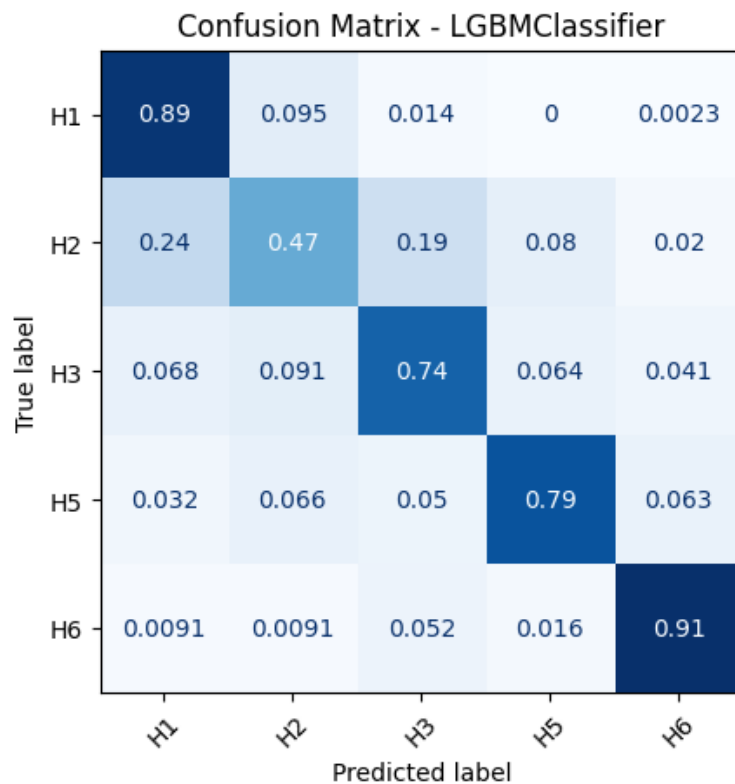
(Figure 2. Cross-Validation vs. Test F1)

Results and Discussion

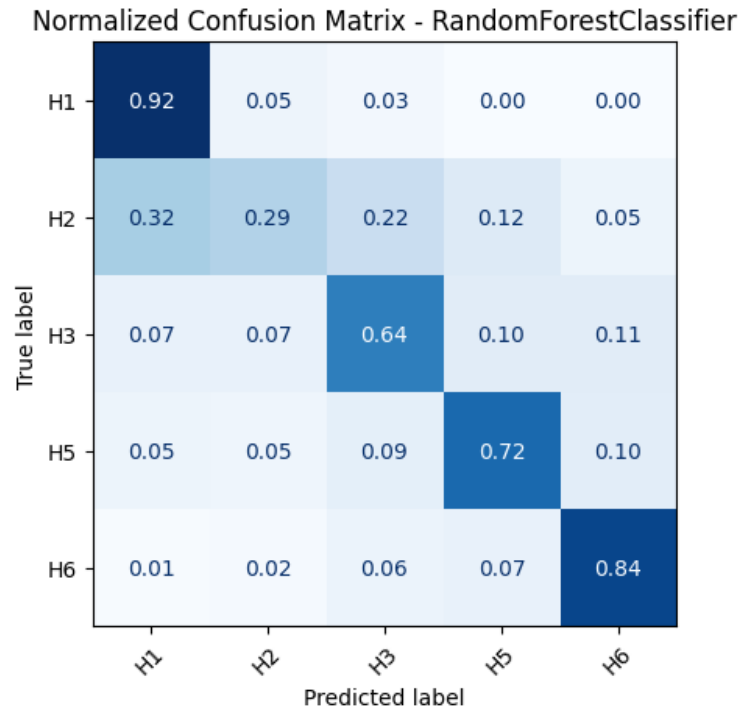
The performance of the tuned models is summarized in Figure 2, which serves as the primary comparison for this analysis. Because the notebook reports cross-validated performance during tuning rather than explicit training-set metrics, the most reliable “training-side” comparison is the 3-fold cross-validated weighted F1-score from RandomizedSearchCV, paired with final test-set results. This approach provides a consistent view of both model selection performance and generalization to unseen data.

Among the evaluated models, LGBMClassifier achieved the strongest overall performance. LightGBM is a gradient boosting algorithm that builds many small decision trees sequentially, where each new tree focuses on correcting the errors of the previous ones. This iterative learning process allows the model to capture complex nonlinear patterns in the data. It produced the highest cross-validated weighted F1-score during training and the best results on the test set, with approximately 76% accuracy and a weighted F1-score near 0.75. XGBClassifier, which also uses gradient boosting but includes additional regularization and optimization techniques, ranked second and demonstrated strong performance.

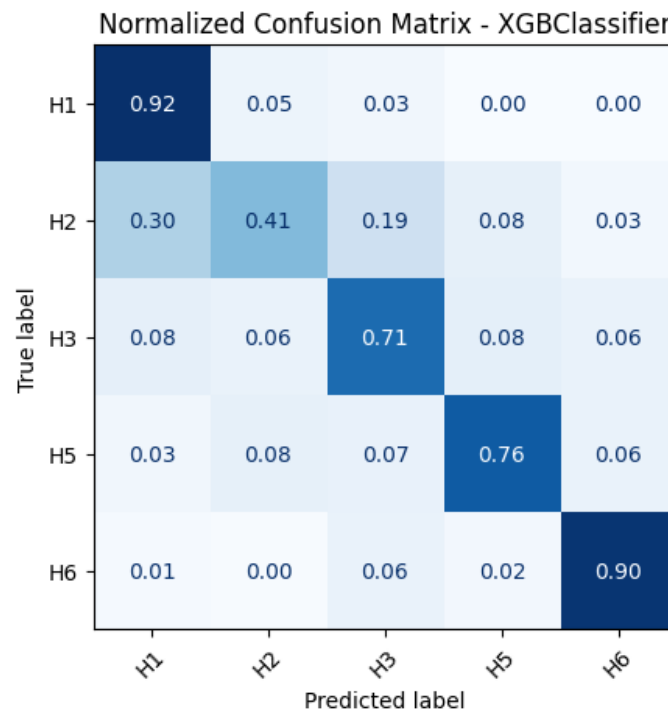
RandomForestClassifier formed a middle tier; it works by training many independent decision trees on random subsets of the data and averaging their predictions to reduce overfitting. The remaining models, DecisionTreeClassifier, BaggingClassifier, and NearestCentroid, performed noticeably worse. A single decision tree is prone to overfitting because it learns specific patterns from the training data, while bagging reduces variance by averaging multiple trees but does not improve bias as effectively as boosting. Nearest Centroid, which classifies samples based on their distance to class centers, was too simplistic for this high-dimensional problem and showed the weakest results overall.



(Figure 4. Confusion Matrix – LGBM Classifier)



(Figure 5. Confusion Matrix – Random Forest Classifier)



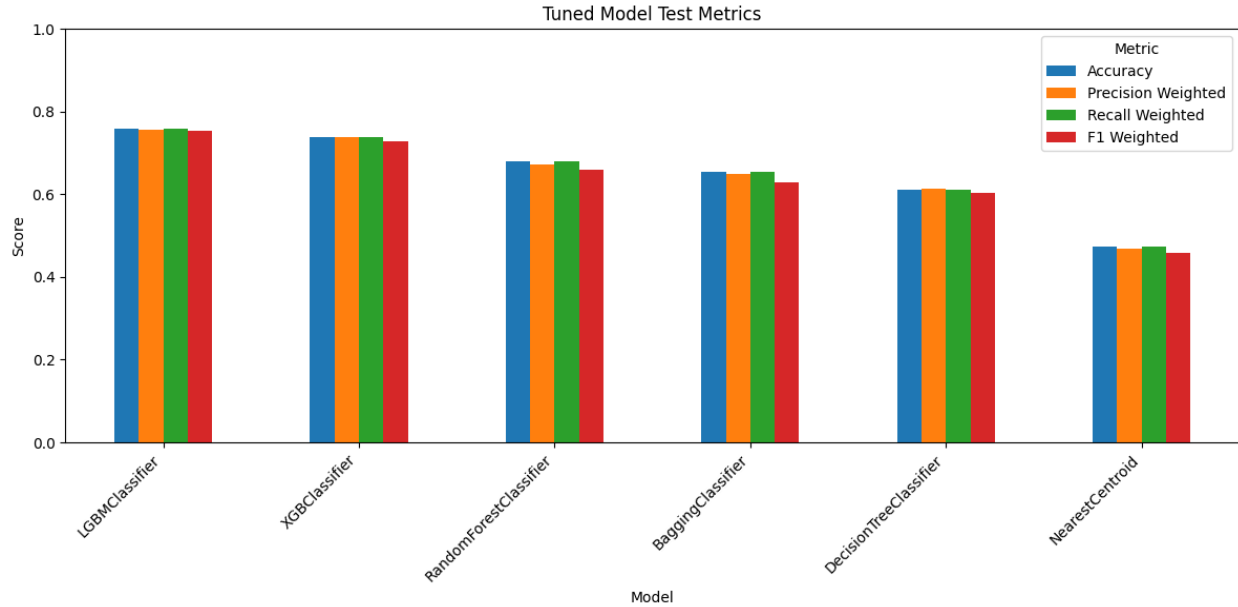
(Figure 6. Confusion Matrix – XGBClassifier)

These findings are supported by the visual summaries generated in the notebook. The tuned test-results table confirms that LightGBM outperformed all other models, while the earlier baseline comparison shows that boosting-based methods were already strong candidates before tuning. In fact, XGBoost ranked first and LightGBM second during the initial screening, indicating early on that ensemble boosting techniques were particularly well suited to this problem due to their ability to model complex feature interactions.

A closer examination of the best-performing model provides additional insight. The LightGBM classification report and confusion matrix reveal that performance varied across classes. The model performed strongly in classes such as H6, achieving high precision, recall, and F1-scores, and showed solid results for H5 and H1. In contrast, H2 proved to be the most challenging class, with significantly lower recall and F1-score, while H3 demonstrated moderate performance. These differences suggest that some classes contain more distinctive patterns, while others overlap more in the feature space, making them harder for the model to separate despite its strong learning capability.

These class-level differences suggest that certain fungal categories are easier to distinguish than others. Strong performance on H6 and H5 likely reflects more distinct visual patterns, even after preprocessing and dimensionality reduction. Conversely, weaker results for H2 indicate overlaps with other classes or less distinctive features. The confusion matrix supports this interpretation, showing that misclassifications are not random but concentrated among visually similar classes.

One of the most important findings is that boosting-based models clearly outperformed simpler approaches. Both LightGBM and XGBoost surpassed Decision Trees, Bagging, and Nearest Centroid by a significant margin. This is expected given the high-dimensional feature space and the presence of complex, nonlinear relationships in the data. Even after PCA reduced the feature space from 3,072 to 425 components, the classification task remained challenging, requiring models capable of capturing intricate patterns. Boosting methods excel in this setting by iteratively correcting errors and refining decision boundaries.



(Figure 2. Tuned Model Test Metrics)

Hyperparameter tuning provided some improvement, but its impact was relatively modest compared to the choice of model. For example, LightGBM showed only a slight increase in weighted F1-score after tuning compared to its baseline performance. This suggests that selecting an appropriate model family had a greater effect on performance than fine-tuning hyperparameters within a limited search space. Additionally, some models did not improve after tuning and, in a few cases, performed slightly worse. This may indicate that the search ranges were too narrow or that certain configurations overfit the cross-validation folds without improving generalization.

Several limitations should be considered when interpreting these results. First, the use of flattened image data removes spatial structure, making it more difficult for traditional machine learning models to capture meaningful visual patterns. While PCA reduced redundancy and improved efficiency, it cannot fully preserve the spatial relationships that convolutional neural networks (CNNs) are designed to exploit. Second, data augmentation was applied before splitting the dataset, which may have introduced similar images across training and test sets, potentially inflating performance estimates. Third, although a second, more focused LightGBM search identified improved cross-validation performance, the refined model was not fully incorporated into the final evaluation, suggesting that the reported results may not reflect the absolute best configuration explored.

Despite these limitations, the results demonstrate that classical machine learning methods can achieve solid performance on these multiclass fungi classification tasks. LightGBM emerged as the most effective model, providing the best balance of accuracy, precision, recall, and weighted F1-score. XGBoost further supports the conclusion that boosting-based ensembles are

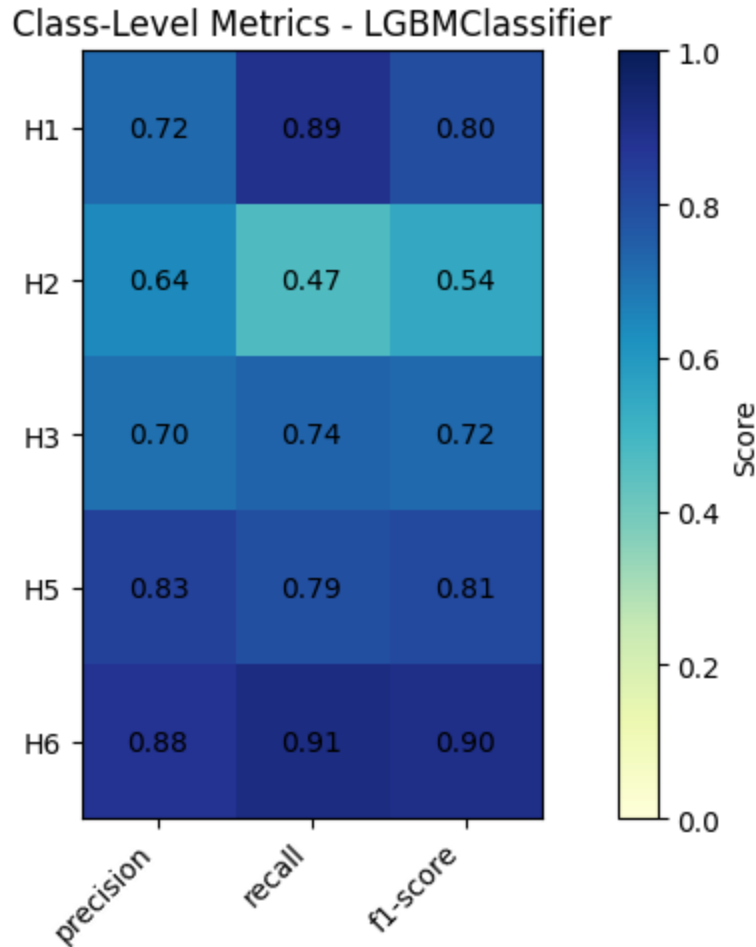
well suited to this problem. At the same time, lower performance in certain classes and constraints in the feature representation highlight opportunities for future improvement. Overall, the findings show that automated fungi classification using traditional machine learning is both feasible and effective, while also pointing toward potential gains from more advanced image-based approaches and improved experimental design.

Model	Training CV F1 Weighted	Test Accuracy	Test Precision Weighted	Test Recall Weighted	Test F1 Weighted
LGBMClassifier	0.7339	0.7598	0.7556	0.7598	0.753
XGBClassifier	0.7113	0.7398	0.7387	0.7398	0.7288
RandomForestClassifier	0.6448	0.6798	0.6716	0.6798	0.6599
BaggingClassifier	0.6177	0.6553	0.6499	0.6553	0.6284
DecisionTreeClassifier	0.5777	0.6122	0.6135	0.6122	0.6036
NearestCentroid	0.4638	0.4737	0.4699	0.4737	0.4587

(Table 2. CV F1 Weighted Table)

Conclusion

This project demonstrates that machine learning can effectively classify microscopic fungi images from the DeFungi dataset. The primary objective, to develop and evaluate a model capable of assigning images to one of five classes (H1, H2, H3, H5, and H6), was successfully achieved. After preprocessing the data, balancing the classes through augmentation, scaling features, and reducing dimensionality with PCA, multiple models were trained and compared. Among the tuned models, LGBMClassifier delivered the best overall performance, achieving approximately 76% test accuracy and a weighted F1-score of about 0.75. These results indicate that the dataset contains sufficient visual information for machine learning models to learn meaningful patterns, even when using traditional methods.



(Figure 3. Class-Level Metrics – LGBM Classifier)

The project also addressed the broader question of whether classical machine learning techniques, combined with image preprocessing and dimensionality reduction, can perform well on a multiclass fungi classification task. The findings suggest that they can. Boosting-based ensemble models, particularly LightGBM and XGBoost, consistently outperformed simpler approaches such as single decision trees and nearest centroid classifiers. This highlights the importance of flexible, nonlinear models for handling high-dimensional, image-derived features. However, class-level results showed uneven performance across categories: some classes, such as H6, were classified accurately, while others, especially H2, remained challenging. This reflects underlying class overlap and subtle visual differences that limit performance.

Several key insights emerged from the project. Class imbalance in the original dataset required careful handling, making augmentation and class weighting essential. Dimensionality reduction with PCA proved critical for managing the large number of features generated by flattened images, reducing redundancy while preserving most of the information. The strongest

performance came from boosting-based models, reinforcing their effectiveness in capturing complex patterns. In contrast, hyperparameter tuning provided only modest gains, suggesting that model selection had a greater impact than fine-tuning within a limited search space.

Despite these positive results, there are clear opportunities for improvement. Future work could explore convolutional neural networks (CNNs), which better preserve spatial relationships in images and may significantly improve classification performance. Refining the preprocessing pipeline, such as applying augmentation only after splitting the dataset, would reduce the risk of data leakage and provide a more rigorous evaluation. Expanding the dataset, particularly for underrepresented or difficult classes, could further improve generalization. Additional approaches, such as extracting texture-based features or testing alternative dimensionality reduction techniques, may also enhance class separation.

In summary, this project confirms that machine learning is a practical and effective approach for fungi image classification. While performance is not perfect, the results are strong enough to validate the methodology and highlight its potential. The project successfully met its objective by developing, comparing, and evaluating multiple models and identifying the best-performing approach for the DeFungi dataset, while also outlining clear directions for future improvement.

References:

Hajati, Farshid . "{DeFungi: Direct Mycological Examination of Microscopic Fungi Images." UC Irvine Machine Learning Repository, 2021, archive.ics.uci.edu/dataset/773/defungi.

Pandala, Shankar Rao. "Shankarpandala/Lazypredict." GitHub, 10 Jan. 2024, github.com/shankarpandala/lazypredict.

scikit-learn. "Sklearn.metrics.confusion_matrix — Scikit-Learn 0.21.3 Documentation." Scikit-Learn.org, 2019, scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html.

"Python API Reference — Xgboost 1.7.5 Documentation." Xgboost.readthedocs.io, xgboost.readthedocs.io/en/stable/python/python_api.html#xgboost.XGBClassifier.

"Lightgbm.LGBMClassifier — LightGBM 3.3.2.99 Documentation." Lightgbm.readthedocs.io, lightgbm.readthedocs.io/en/latest/pythonapi/lightgbm.LGBMClassifier.html.

scikit-learn. "Sklearn.model_selection.RandomizedSearchCV — Scikit-Learn 0.21.3 Documentation." Scikit-Learn.org, 2019, scikit-learn.org/stable/modules/generated/sklearn.model_selection.RandomizedSearchCV.html.

"PCA." Scikit-Learn, 2025, scikit-learn.org/1.7/modules/generated/sklearn.decomposition.PCA.html. Accessed 4 May 2026.

scikit-learn. "StandardScaler." Scikit-Learn.org, 2019, scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html.